



Fall 2025 – Semester Project

Student Name: Muhammad Mohsin

CMS ID: 540446

Class: BSCS-15-C

Project: Renewable Energy Microgrid Battery Scheduler (Terminal-based)

The code for the program is:

```
#include<iostream>

#include<iomanip> //for file formatting
#include<fstream> //for file handling
#include<cmath>   //for mathematical functions (fmin, fmax)

using namespace std;

const int HOURS = 24; //for 24 hours of the day
const int DEVICES = 5; // I take 5 devices, for example
const int QUARTERS = 4; //four quarters of the hour
const float BATTERY_CAP = 2220.0; //180 Ah - for 12 V = 2220 Wh.

enum State {CHARGE, DISCHARGE, IDLE, BLACKOUT}; //added enum for different states of
battery.

int main()
{
    float solar[HOURS]; //Arrays to input devices and hourly data.
    float demand[HOURS][DEVICES]; //2D array for device demand over 24 hours.

    ifstream infile("solar_input.txt"); //take input of data from file
    if (!infile) // if file has errors/file does not exist:
    {
        cout << "ERROR: Cannot open solar_input.txt\n";
```



```
    cout << "Make sure the file is in the same folder.\n";
    return 1;
}

for (int i = 0; i < HOURS; i++) // Read 24 solar values, spread over lines in
the file
{
    infile >> solar[i];
}

int base_load[DEVICES];

for (int d = 0; d < DEVICES; d++) // Read 5 device loads (they are going to be
same for all hours)
{
    infile >> base_load[d];
}

for (int h = 0; h < HOURS; h++) // Copy base load to all 24 hours
{
    for (int d = 0; d < DEVICES; d++)
    {
        demand[h][d] = base_load[d];
    }
}

infile.close();

ofstream outfile("simulation_report.txt"); //create output file
if (!outfile) // if file creation has errors
{
    cout << "ERROR: Cannot create simulation_report.txt\n"; //file creation error
```



```
        return 1;
    }

    float charge_state = 30.0; //at 0700 or 0800 hrs the charge state is roundabout
    ~30%.

    int blackout_quads = 0; //if battery percentage is less than 30%, add 1 to
    blackout counter.

    cout << fixed << setprecision(1); //set decimal precision to 1
    place.

    outfile << fixed << setprecision(1); //for output file
    cout << "\n=== MICROGRID 24-HOUR SIMULATION ===\n\n";
    outfile << "\n=== MICROGRID 24-HOUR SIMULATION ===\n\n"; //for output file
    cout << "Time      Solar(W)   Load(W)   Net(W)     Action      SOC(%)\\n";
    outfile << "Time      Solar(W)   Load(W)   Net(W)     Action      SOC(%)\\n";
    //for output file
    cout << "-----\\n";
    outfile << "-----\\n";
    //for output file

    for (int hour = 0; hour < HOURS; hour++)
    {
        float hour_solar = solar[hour];
        float hour_load = 0;

        for (int d = 0; d < DEVICES; d++) // Add load of all 5 devices for this
        particular hour
        {
            hour_load += demand[hour][d];
        }

        for(int quarter = 0; quarter < QUARTERS; quarter++)
        {
            //Core Logic here to calculate net charge, state of charge, action, etc.
            float quarter_solar = hour_solar / 4.0;
```



```
float quarter_load = hour_load / 4.0;

float net_charge = quarter_solar - quarter_load; //remaining charge after
solar power produced minus the power usage that quarter.

State action;

if (net_charge > 0 && charge_state < 100.0)
{
    // CHARGE: extra solar power available and battery is not full
    action = CHARGE;
    float space_in_battery = (100.0 - charge_state) / 100.0 *
BATTERY_CAP;
    float available_energy = net_charge * 0.25; // 15 min
= 0.25 hours
    float actual_charge = fmin(available_energy, space_in_battery);
    charge_state += (actual_charge / BATTERY_CAP) * 100.0;
}
else if (net_charge < 0 && charge_state > 30.0)
{
    // DISCHARGE: need power and battery has enough power
    action = DISCHARGE;
    float energy_needed = -net_charge * 0.25; //negative net charge means
we need this much energy
    float available_energy = (charge_state - 30.0) / 100.0 * BATTERY_CAP;
    float actual_discharge = fmin(energy_needed, available_energy);
    charge_state -= (actual_discharge / BATTERY_CAP) * 100.0;
}
else if (net_charge < 0 && charge_state <= 30.0)
{
    // BLACKOUT: need power but battery is too low
    action = BLACKOUT;
    blackout_quads++;
}
else
{

```



```
// IDLE: balanced or no activity
    action = IDLE;
}

    cout << setw(2) << hour << ":" << setw(2) << (quarter * 15) << " " //
Display one line per quarter-hour
    << setw(8) << quarter_solar << " "
    << setw(8) << quarter_load << " "
    << setw(8) << net_charge << " ";

    outfile << setw(2) << hour << ":" << setw(2) << (quarter * 15) <<
" " //for output file
    << setw(8) << quarter_solar << " "
    << setw(8) << quarter_load << " "
    << setw(8) << net_charge << " ";

string action_str;
    if (action == CHARGE)
        action_str = "CHARGE ";
    else if (action == DISCHARGE)
        action_str = "DISCHARGE";
    else if (action == BLACKOUT)
        action_str = "BLACKOUT ";
    else
        action_str = "IDLE ";

    cout << action_str << " " << setw(6) << charge_state << "%\n"; // Print
to screen
    outfile << action_str << " " << setw(6) << charge_state << "%\n"; //
Print to file
}
}
```



```
int uptime_hours = HOURS - (blackout_quads / QUARTERS);

cout << "-----\n";
outfile << "-----\n";
//for output file
cout << "Simulation complete!\n";
outfile << "Simulation complete!\n"; //for output file
cout << "Final battery level: " << charge_state << "%\n";
outfile << "Final battery level: " << charge_state << "%\n"; //for output file
cout << "Uptime: " << uptime_hours << " out of 24 hours\n";
outfile << "Uptime: " << uptime_hours << " out of 24 hours\n"; //for output file

if (uptime_hours == 24)
{
    cout << "SUCCESS: 100% uptime achieved!\n";
    outfile << "SUCCESS: 100% uptime achieved!\n"; //for output file
}
else
{
    cout << "Blackouts occurred in " << (blackout_quads / QUARTERS) << "
hours.\n";
    outfile << "Blackouts occurred in " << (blackout_quads / QUARTERS) << "
hours.\n"; //for output file
}
outfile.close(); //Closing the file
cout << "\nReport saved to 'simulation_report.txt'\n";
return 0;
}
```

Screenshot of the output code is:



National University of Sciences & Technology (NUST)
School of Electrical Engineering and Computer Science (SEECS)
Department of Computing



```
==== MICROGRID 24-HOUR SIMULATION ====
```

Time	Solar(W)	Load(W)	Net(W)	Action	SOC(%)
0: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
0:15	0.0	72.5	-72.5	BLACKOUT	30.0%
0:30	0.0	72.5	-72.5	BLACKOUT	30.0%
0:45	0.0	72.5	-72.5	BLACKOUT	30.0%
1: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
1:15	0.0	72.5	-72.5	BLACKOUT	30.0%
1:30	0.0	72.5	-72.5	BLACKOUT	30.0%
1:45	0.0	72.5	-72.5	BLACKOUT	30.0%
2: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
2:15	0.0	72.5	-72.5	BLACKOUT	30.0%
2:30	0.0	72.5	-72.5	BLACKOUT	30.0%
2:45	0.0	72.5	-72.5	BLACKOUT	30.0%
3: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
3:15	0.0	72.5	-72.5	BLACKOUT	30.0%
3:30	0.0	72.5	-72.5	BLACKOUT	30.0%
3:45	0.0	72.5	-72.5	BLACKOUT	30.0%
4: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
4:15	0.0	72.5	-72.5	BLACKOUT	30.0%
4:30	0.0	72.5	-72.5	BLACKOUT	30.0%
4:45	0.0	72.5	-72.5	BLACKOUT	30.0%
5: 0	0.0	72.5	-72.5	BLACKOUT	30.0%
5:15	0.0	72.5	-72.5	BLACKOUT	30.0%
5:30	0.0	72.5	-72.5	BLACKOUT	30.0%
5:45	0.0	72.5	-72.5	BLACKOUT	30.0%
6: 0	55.0	72.5	-17.5	BLACKOUT	30.0%
6:15	55.0	72.5	-17.5	BLACKOUT	30.0%
6:30	55.0	72.5	-17.5	BLACKOUT	30.0%
6:45	55.0	72.5	-17.5	BLACKOUT	30.0%
7: 0	220.0	72.5	147.5	CHARGE	31.7%
7:15	220.0	72.5	147.5	CHARGE	33.3%
7:30	220.0	72.5	147.5	CHARGE	35.0%
7:45	220.0	72.5	147.5	CHARGE	36.6%
8: 0	230.0	72.5	157.5	CHARGE	38.4%
8:15	230.0	72.5	157.5	CHARGE	40.2%
8:30	230.0	72.5	157.5	CHARGE	42.0%
8:45	230.0	72.5	157.5	CHARGE	43.7%
9: 0	230.0	72.5	157.5	CHARGE	45.5%
9:15	230.0	72.5	157.5	CHARGE	47.3%
9:30	230.0	72.5	157.5	CHARGE	49.1%
9:45	230.0	72.5	157.5	CHARGE	50.8%
10: 0	255.0	72.5	182.5	CHARGE	52.9%
10:15	255.0	72.5	182.5	CHARGE	54.9%
10:30	255.0	72.5	182.5	CHARGE	57.0%
10:45	255.0	72.5	182.5	CHARGE	59.1%
11: 0	295.0	72.5	222.5	CHARGE	61.6%
11:15	295.0	72.5	222.5	CHARGE	64.1%
11:30	295.0	72.5	222.5	CHARGE	66.6%
11:45	295.0	72.5	222.5	CHARGE	69.1%
12: 0	350.0	72.5	277.5	CHARGE	72.2%

12:15	350.0	72.5	277.5	CHARGE	75.3%
12:30	350.0	72.5	277.5	CHARGE	78.5%
12:45	350.0	72.5	277.5	CHARGE	81.6%
13: 0	345.0	72.5	272.5	CHARGE	84.6%
13:15	345.0	72.5	272.5	CHARGE	87.7%
13:30	345.0	72.5	272.5	CHARGE	90.8%
13:45	345.0	72.5	272.5	CHARGE	93.9%
14: 0	322.5	72.5	250.0	CHARGE	96.7%
14:15	322.5	72.5	250.0	CHARGE	99.5%
14:30	322.5	72.5	250.0	CHARGE	100.0%
14:45	322.5	72.5	250.0	IDLE	100.0%
15: 0	400.0	72.5	327.5	IDLE	100.0%
15:15	400.0	72.5	327.5	IDLE	100.0%
15:30	400.0	72.5	327.5	IDLE	100.0%
15:45	400.0	72.5	327.5	IDLE	100.0%
16: 0	295.0	72.5	222.5	IDLE	100.0%
16:15	295.0	72.5	222.5	IDLE	100.0%
16:30	295.0	72.5	222.5	IDLE	100.0%
16:45	295.0	72.5	222.5	IDLE	100.0%
17: 0	2425.0	72.5	2352.5	IDLE	100.0%
17:15	2425.0	72.5	2352.5	IDLE	100.0%
17:30	2425.0	72.5	2352.5	IDLE	100.0%
17:45	2425.0	72.5	2352.5	IDLE	100.0%
18: 0	575.0	72.5	502.5	IDLE	100.0%
18:15	575.0	72.5	502.5	IDLE	100.0%
18:30	575.0	72.5	502.5	IDLE	100.0%
18:45	575.0	72.5	502.5	IDLE	100.0%
19: 0	200.0	72.5	127.5	IDLE	100.0%
19:15	200.0	72.5	127.5	IDLE	100.0%
19:30	200.0	72.5	127.5	IDLE	100.0%
19:45	200.0	72.5	127.5	IDLE	100.0%
20: 0	50.0	72.5	-22.5	DISCHARGE	99.7%
20:15	50.0	72.5	-22.5	DISCHARGE	99.5%
20:30	50.0	72.5	-22.5	DISCHARGE	99.2%
20:45	50.0	72.5	-22.5	DISCHARGE	99.0%
21: 0	25.0	72.5	-47.5	DISCHARGE	98.5%
21:15	25.0	72.5	-47.5	DISCHARGE	97.9%
21:30	25.0	72.5	-47.5	DISCHARGE	97.4%
21:45	25.0	72.5	-47.5	DISCHARGE	96.8%
22: 0	150.0	72.5	77.5	CHARGE	97.7%
22:15	150.0	72.5	77.5	CHARGE	98.6%
22:30	150.0	72.5	77.5	CHARGE	99.5%
22:45	150.0	72.5	77.5	CHARGE	100.0%
23: 0	5.0	72.5	-67.5	DISCHARGE	99.2%
23:15	5.0	72.5	-67.5	DISCHARGE	98.5%
23:30	5.0	72.5	-67.5	DISCHARGE	97.7%
23:45	5.0	72.5	-67.5	DISCHARGE	97.0%

```
-----
```

Simulation complete!
Final battery level: 97.0%
Uptime: 17 out of 24 hours
Blackouts occurred in 7 hours.

A .txt file is used and printed out.